1.0 Engineering Problem

Quality inspection of packaged pork rashers requires packaging and product-placement defects to be identified before distribution. This project investigates whether a computer-vision object-detection system can reliably detect and localise **loose meat, twisted meat, unsealed packaging and packaging errors** in RGB pork-rasher tray images, producing the defect class, confidence score and location required for rejection or manual review [1], [2].

For this application, **false negatives are more critical than false positives**, because a missed defect may allow a defective package to pass inspection, whereas a false positive generally results in unnecessary rejection or review. **Recall** is therefore given particular importance, while precision is monitored to limit false alarms.

The final scope contains four defect classes; the original wrinkle class was excluded following dataset auditing, as discussed in Section 2.1. The project is limited to the supplied image datasets and therefore evaluates a prototype inspection method rather than production-line readiness.

2.0 Inputs, Outputs and Data

The supplied dataset consists of RGB JPEG images of packaged pork-rasher trays at 720 × 540 pixels. During model development, labelled images are accompanied by YOLO-format annotations specifying the defect class and normalised bounding-box location. A dataset configuration file defines the training, validation and test image paths and four target classes, while operational inference accepts a single RGB pork-rasher tray image processed at **640 pixels**.

For each detected object, the system produces three **outputs**:

1. **Predicted defect class:** loose meat, packaging error, twisted meat or unsealed packaging.
2. **Detection confidence:** a value between 0 and 1 representing model confidence.
3. **Bounding-box coordinates:** pixel coordinates (x_1, y_1, x_2, y_2) defining the detected region.

For the selected inspection model, detections are reported when confidence is at least **0.201**. This operating threshold was selected using validation data to favour defect recall, because missed defects were considered more critical than additional false-positive inspections. It is an inference threshold and is separate from the lower confidence setting used when calculating validation metrics such as mAP.

Validation: Model predictions are compared with withheld labelled images containing known defect classes and ground-truth bounding boxes. Validation data are used for model selection, while the independent test set is used to assess performance on unseen data.

Missing image files, missing model weights, unreadable or corrupted images, and invalid confidence values are rejected before inference and reported as input errors. Images outside the validated pork-rasher inspection domain may still produce predictions, but these outputs are considered unsupported because the model has not been validated for those conditions and should not be used for automated decisions without further validation.

The outputs are intended to support automated pre-screening by flagging potentially defective packages for rejection or further manual review.

2.1 Data Preparation and Audit

Before model development, the supplied datasets were audited to reduce bias and invalid evaluation. Training data were used to fit model parameters, validation data to compare configurations, and the test set was preserved for final evaluation on unseen data.

Some augmented images originating from the same source frame appeared across different dataset splits. These overlaps were identified using the filename source identifier and removed while preserving the test split. A reusable Python dataset-preparation module was applied consistently across Dataset Versions 1, 2 and 4 to perform source-overlap removal and annotation auditing. Images with empty annotations were retained when appropriate because an image containing no target defect represents a valid background or negative example for object detection. The cleaned Version 1 dataset distribution is in Table 2.1.

Table 2.1. Cleaned Version 1 dataset distribution

Split	Images	Background	Loose meat	Packaging error	Twisted meat	Unsealed
Training	1,935	29	139	931	91	1,107
Validation	119	0	5	57	5	87
Test	80	1	3	37	7	52

The dataset was strongly class-imbalanced, with substantially fewer loose-meat and twisted-meat examples than packaging-error and unsealed examples. Class-specific metrics were therefore examined alongside overall performance.

Differences were also identified between dataset versions. Versions 2 and 4 used centre-cropped preprocessing, which removed portions of some labelled defects, whereas Version 1 preserved more of the complete tray through fitted-image preprocessing. Because the task requires detection and localisation of defect patterns, cropping can remove visual features and surrounding context needed to distinguish a defect from normal packaging or meat. Version 1 was therefore retained for later development.

The original dataset also contained a wrinkle class. However, V4 provided only a small number of independent wrinkle examples across the training and validation splits and no labelled wrinkles in the test split. Centre cropping also truncated a wrinkle annotation in an inspected validation image. Consequently, unseen wrinkle performance could not be evaluated defensibly, and the class was excluded from the final project scope.

3.0 Computational Solution

The computational solution was developed using the Ultralytics YOLOv8 object-detection framework. YOLO performs single-stage object detection, predicting defect classes and bounding-box locations directly from image features within one inference pipeline. YOLOv8n was selected as the baseline architecture because it provides relatively low computational cost while supporting multi-class object detection. A larger YOLOv8s model was subsequently evaluated as an alternative in Section 4 [3], [4].

Model development was implemented in Python using Ultralytics YOLO in Google Colab. Development began with a YOLOv8n baseline trained for 30 epochs. Subsequent optimisation investigated four main factors: mosaic augmentation, input resolution, learning

rate and training duration. AdamW was used consistently in the controlled learning-rate experiments so that the effects of learning rate and training duration could be compared under the same optimiser.

Validation performance, particularly recall and mAP50, was used to decide whether each configuration should be retained. The final validation-selected model used the cleaned Version 1 dataset, YOLOv8n architecture, 640-pixel input resolution and disabled mosaic augmentation. The selected model configuration is summarised in Table 3.1.

Table 3.1. Selected model configuration

Setting	Selected value
Dataset	Cleaned Version 1
Architecture	YOLOv8n
Input resolution	640 px
Mosaic augmentation	Disabled
Epochs	30
Batch size	16
Random seed	42
Inference confidence threshold	0.201

The complete computational workflow is summarised in Figure 3.1. During inference, a new RGB pork-rasher image is passed through the trained model, which returns the predicted defect class, confidence score and bounding-box coordinates for each reported detection.

The approach assumes that future inspection images are reasonably similar to the supplied dataset in camera view, packaging appearance and defect presentation, and that the supplied bounding-box annotations provide sufficiently accurate ground truth for training and validation.

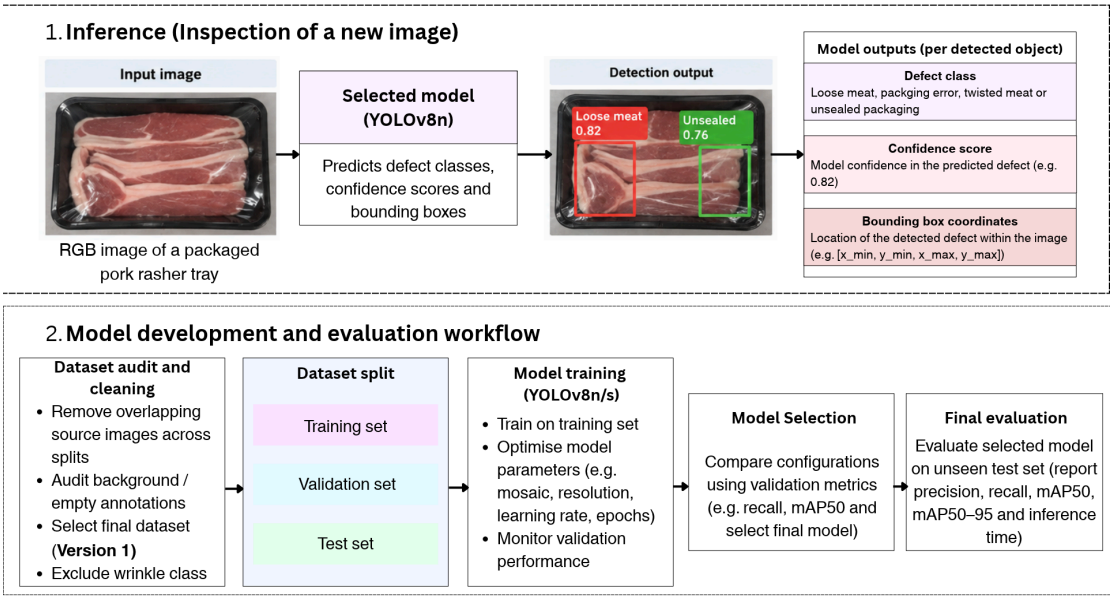


Figure 3.1 Computational workflow for pork-rasher defect detection, showing the operational inference pipeline and the model-development, selection and evaluation process.

4.0 Alternative Solutions

Two alternative approaches were investigated before selecting the final development pathway: increasing model capacity from YOLOv8n to YOLOv8s, and changing the dataset preprocessing from centre-cropped images to fitted images that preserved more of the original tray context.

4.1 Model Architecture: YOLOv8n vs YOLOv8s

A larger **YOLOv8s** model was evaluated against the YOLOv8n baseline using the same cleaned Version 2 dataset and comparable training settings. The purpose was to determine whether the additional model capacity produced sufficient detection improvement to justify its higher computational cost.

YOLOv8s is substantially larger than YOLOv8n. At a 640-pixel input resolution, YOLOv8n contains approximately **3.2 million parameters and requires 8.7 GFLOPs**, whereas YOLOv8s contains approximately **11.2 million parameters and requires 28.6 GFLOPs** [3]. A higher parameter count means that more model weights must be stored and processed, while a higher GFLOP count indicates that more arithmetic operations are required for each forward pass. Consequently, YOLOv8s requires greater computational resources and generally has higher inference latency.

Despite this additional model capacity, YOLOv8s did not improve the two metrics given greatest importance in this project. Validation **precision** decreased from **63.2% to 52.3%**, while **recall** decreased slightly from **47.8% to 47.1%**. YOLOv8s increased mAP50 from **51.9% to 54.8%** and detected more packaging errors, but mAP50–95 decreased from **30.8% to 27.3%**, twisted-meat recall remained zero, and unmatched false-positive detections increased from **20 to 41**.

The larger model therefore introduced substantially greater computational cost without improving either overall precision or recall. The small gain in mAP50 was not considered sufficient to justify the additional model complexity and false-positive detections. YOLOv8n was therefore retained as the main architecture because it provided a more favourable balance between detection performance and computational efficiency for a potential real-time inspection application.

Detailed YOLOv8n and YOLOv8s validation results and confusion-matrix comparisons are provided in **Appendix B.5**.

4.2 Image Preprocessing: Centre Crop vs Fitted Images

A second alternative investigated the effect of dataset preprocessing. Dataset Version 2 used **centre-cropped images**, whereas Version 1 used **fitted-image preprocessing** that preserved more of the complete tray and surrounding defect context.

Visual inspection showed that centre cropping could remove portions of labelled defects near image boundaries. Because the model is required to recognise and localise defect patterns rather than simply identify the presence of pork, removing part of a defect can also remove visual features and surrounding context needed to distinguish defective regions from normal meat or packaging.

Version 1 was retained primarily because its fitted-image preprocessing preserved more complete defect and tray context. For comparison, the Version 1 YOLOv8n baseline later achieved higher test recall than Version 2, increasing from **43.9% to 51.8%**, while mAP50 increased from **49.9% to 58.2%**. Version 1 also detected some twisted-meat instances, achieving **28.6% twisted-meat recall**, whereas Version 2 detected none.

Because the two exported dataset versions contained different annotation counts and preprocessing histories, these test results are treated as supporting observations rather than a controlled model-selection criterion. The preserved defect context provided the primary justification for continuing development with Version 1.

Detailed Version 2 preprocessing observations are provided in **Appendix B.6**, while the subsequent Version 1 development and optimisation results are provided in **Appendix C**.

Overall, these comparisons showed that greater model complexity did not automatically improve inspection performance, while preserving relevant image context had a stronger influence on the subsequent development pathway. The final approach therefore retained **YOLOv8n** and continued optimisation using **Dataset Version 1**.

5.0 Validation

Model performance was evaluated using labelled validation and test images containing known defect classes and ground-truth bounding boxes. During Version 1 optimization, the **validation set** was used to compare model configurations, select the operating confidence threshold and guide final model selection. After the final configuration was selected, it was assessed on the **Version 1 test set** to evaluate performance on unseen data. Earlier baseline test comparisons between dataset versions were treated as supporting observations and were not used to tune the final no-mosaic configuration.

Validation considered both **detection accuracy and localisation quality**. Because missed defects are considered more critical than false alarms in this application, recall was given particular importance. Precision was monitored to assess unnecessary detections, while mean Average Precision (mAP) was used to evaluate detection and localisation performance across confidence thresholds.

F1-confidence curves were also examined when selecting the operational confidence threshold, as they show the trade-off between precision and recall at different thresholds. Visual inspection of predictions and confusion matrices was used alongside numerical metrics to identify class-specific failure modes, which was particularly important because the dataset is strongly class-imbalanced and some classes contain only a small number of validation or test examples.

No validated industrial acceptance threshold was provided for this application. The reported metrics are therefore used for comparative evaluation and engineering judgement rather than as evidence of production certification.

5.1 Performance Metrics

$$Precision = \frac{TP}{TP + FP}$$

Precision measures the proportion of predicted defects that are correct. Higher precision therefore corresponds to fewer false-positive detections.

$$Recall = \frac{TP}{TP + FN}$$

Recall measures the proportion of actual defects detected by the model. Recall is especially important in this project because a false negative may allow a defective package to pass inspection.

$$IoU = \frac{Area\ of\ Intersection}{Area\ of\ Union}$$

Intersection over Union (IoU) measures the spatial overlap between a predicted bounding box and its ground-truth annotation [5].

mAP50 evaluates average detection performance when a predicted bounding box is considered correct at an IoU threshold of 0.50 [5].

mAP50–95 averages performance over IoU thresholds from 0.50 to 0.95 and therefore provides a more demanding measure of detection and localisation quality [5].

5.2 Final Test Performance

The selected **YOLOv8n, 640-pixel, no-mosaic model** was evaluated on the held-out test set of **80 images containing 99 labelled defects**. It achieved **52.8% precision, 52.4% recall, 54.4% mAP50 and 22.6% mAP50–95**.

These results were lower than the corresponding validation results of **64.1% precision, 62.3% recall, 69.5% mAP50 and 32.1% mAP50–95**, indicating that the validation improvement did not fully generalise to unseen test data.

Performance also varied substantially between defect classes. The Ultralytics evaluation summary reported class-specific recalls of **73.1% for unsealed packaging, 66.7% for loose meat, 45.9% for packaging errors and 23.9% for twisted meat**. Unsealed packaging was therefore the most consistently detected class, while twisted meat remained the most difficult. However, the test set contained only **seven twisted-meat instances**, so this class-specific result should be interpreted cautiously.

From an engineering perspective, an overall recall of 52.4% means that a substantial proportion of labelled defects were still missed. Since false negatives are considered more critical than false positives, the model demonstrates a functioning inspection prototype but does not provide sufficient evidence for autonomous production-line acceptance or rejection. Further improvement would require more representative training data, particularly for the underrepresented classes, together with additional independent validation.

Detailed validation comparisons, class-specific results and model-selection evidence are provided in **Appendix C**.

5.3 Error Analysis

Figure 5.1 Test-set confusion matrix for the selected YOLOv8n no-mosaic model at a confidence threshold of 0.201, where rows represent predicted classes and columns represent true classes. At this threshold, the model correctly detected **41 of 52 unsealed defects** and **2 of 3 loose-meat defects**, but only **15 of 37 packaging errors** and **1 of 7 twisted-meat defects**.

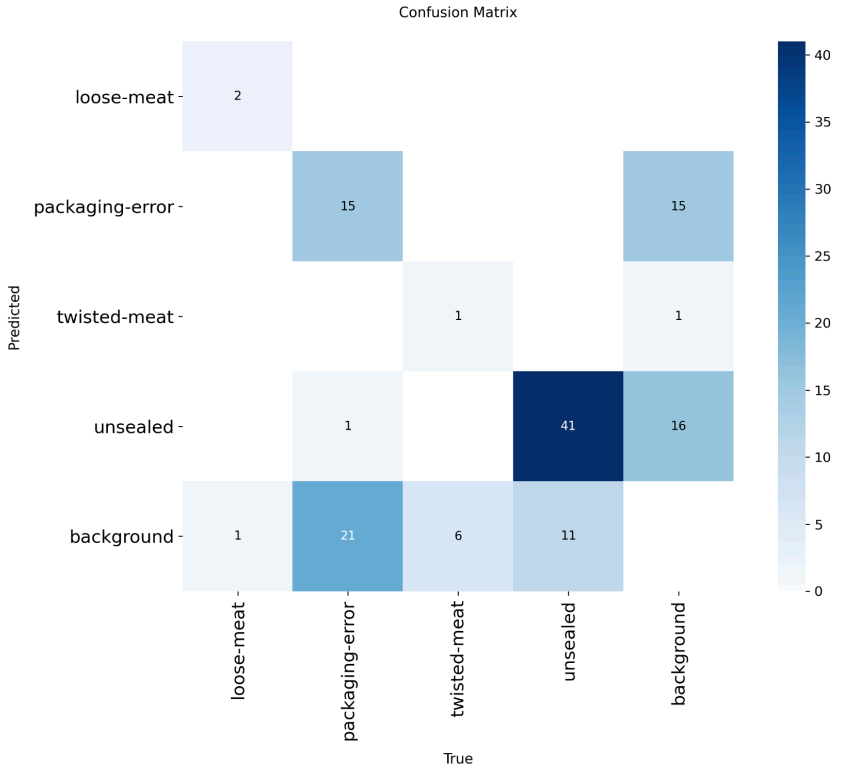


Figure 5.1: Test-set confusion matrix for the selected model at confidence = 0.201.

The main false-negative problems were therefore packaging errors and twisted meat, with **21 packaging errors** and **6 twisted-meat defects** missed. This is consistent with earlier development observations that the less represented and visually difficult classes were less reliably detected. The small number of twisted-meat test examples also means that its class-specific performance should be interpreted cautiously.

The matrix also shows **15 unmatched packaging-error predictions, 16 unmatched unsealed predictions and one unmatched twisted-meat prediction** on background regions. This reflects the trade-off introduced by the relatively low confidence threshold: more potential defects are retained, but additional false alarms are introduced. The threshold of 0.201 was selected using validation data, while the mAP results in Section 5.2 evaluate performance across confidence thresholds.

From an engineering perspective, the missed packaging-error and twisted-meat defects remain the most important limitation because false negatives may allow defective packages to pass inspection. The current model is therefore more suitable as a **pre-screening or decision-support tool** than as a fully autonomous acceptance/rejection system.

Detailed validation results, class-specific metrics and optimisation comparisons are provided in **Appendix C**.

6.0 Optimisation and Performance

Four main parameters were investigated using the cleaned Version 1 dataset: **mosaic augmentation, input resolution, learning rate and training duration**. Validation results were used to compare configurations and guide optimisation, while test results were not used to tune the final selected configuration. Detailed experimental results are provided in Appendix C.

Disabling **mosaic augmentation** produced the most important improvement for the engineering objective. Mosaic combines multiple training images into composite images,

which can improve generalisation but may also reduce the realism of the original tray context. Compared with the V1 baseline, disabling mosaic increased validation recall from **46.8% to 62.3%** and mAP50 from **68.3% to 69.5%**, although precision decreased from **77.6% to 64.1%**. Because missed defects were considered more critical than false alarms, this trade-off supported retaining the no-mosaic configuration.

Increasing **input resolution** from **640 to 960 pixels** improved packaging-error recall but reduced overall recall and mAP50, so the additional computational cost was not justified. Reducing the learning rate from **0.00125 to 0.0005** improved all four overall metrics within the controlled learning-rate experiment, but neither learning-rate configuration exceeded the original no-mosaic model in recall or mAP50. Extending the lower-learning-rate model from **30 to 50 epochs** slightly improved recall and mAP50 but reduced precision and mAP50–95.

The final **YOLOv8n, 640-pixel, no-mosaic model** was therefore selected because it provided the strongest combination of validation recall and mAP50 among the tested configurations, aligning best with the project priority of reducing missed defects.

6.1 Computational Performance

Computational performance was measured during final test evaluation on an **NVIDIA Tesla T4 GPU**. The selected model required approximately **4.5 ms preprocessing, 5.4 ms inference and 2.9 ms postprocessing per image**, corresponding to approximately **12.8 ms per image** for the measured processing stages, or about **78 images/s**. Inference alone corresponds to approximately **185 images/s**.

The fused YOLOv8n model contained approximately **3.01 million parameters** and required **8.1 GFLOPs** per forward pass, supporting its use as the lower-complexity architecture.

These results indicate that model computation is unlikely to be the main bottleneck for single-image inspection on comparable GPU hardware. However, the measurements exclude camera acquisition, image transfer, production-line control and other deployment overheads. They therefore demonstrate **computational capability** rather than proving that the complete system satisfies a specific industrial throughput requirement.

7.0 Limitations and Lessons Learned

The main limitation of the developed system is the quantity and representativeness of the available data. The dataset was strongly class-imbalanced, and the final test set contained only **7 twisted-meat** and **3 loose-meat** instances, so class-specific results for these minority classes should be interpreted cautiously. The wrinkle class was excluded because it had limited independent examples, no labelled test instances, and preprocessing could truncate the annotated defect region.

The selected model also performed worse on the unseen test set than on validation data, showing that validation performance did not fully represent generalisation. In addition, the model was evaluated only on the supplied dataset and was not tested under different cameras, viewing angles, lighting conditions, packaging variations or production-line environments.

A key lesson was that **greater model complexity did not automatically improve inspection performance**. YOLOv8s, higher resolution and longer training produced mixed trade-offs, whereas dataset preparation and removal of mosaic augmentation had a greater influence on the selected model. The project also demonstrated the importance of preventing

source-image leakage between dataset splits and using validation results for model selection rather than relying on training performance alone.

Overall, the current system should be considered a **pre-screening or decision-support prototype**. Further development would require more representative and balanced defect data, particularly for twisted meat and packaging errors, followed by independent testing under realistic production conditions before autonomous acceptance or rejection could be justified.

8.0 AI Use and Critical Reflection

ChatGPT was used as a supporting tool throughout the project, mainly for **coding guidance, interpretation of model-training results, experiment planning, and improving the professionalism of code comments, documentation and report writing**. Its contribution was moderate: it assisted with troubleshooting and interpretation, but the dataset preparation, model training, evaluation and generation of experimental results were carried out using my own notebooks and project data.

One useful contribution was helping me interpret training and validation results and suggesting possible follow-up experiments. For example, after reviewing the baseline results and the characteristics of the dataset, ChatGPT suggested testing the model with **mosaic augmentation disabled** to determine whether preserving the original tray context would improve defect detection. I implemented and evaluated this experiment myself, and the resulting increase in validation recall supported retaining the no-mosaic configuration. ChatGPT also helped organise comparisons between dataset Versions 1, 2 and 4 and explain metrics such as precision, recall and mAP when interpreting model behaviour.

ChatGPT was also used to improve the clarity and professionalism of **code comments, docstrings, HTML documentation and report wording**. These suggestions were reviewed against the implemented code, notebook outputs and dataset contents before being included. Some AI suggestions required correction when they did not match the specific dataset version, model configuration or recorded experimental result, so outputs were not accepted without verification.

Final technical decisions remained my responsibility. Decisions including excluding the wrinkle class, retaining YOLOv8n, prioritising recall, selecting the 640-pixel no-mosaic model and treating the final system as a pre-screening prototype were based on my experimental results and engineering interpretation. Overall, ChatGPT improved the efficiency of debugging, analysis and documentation, while verification of results and final engineering judgement remained under my control.

References

[1] Hello, “Pork Rasher Error (Packaging) Computer Vision Dataset,” *Roboflow Universe*, object-detection dataset, CC BY 4.0, accessed Oct. 14, 2026. Roboflow pork-rasher dataset

[2] Monash University, “Project Information – Dataset 3: Automated Detection of Pork Rasher Packaging Errors and Meat Quality,” *MMA3001 Project Datasets*, 2026. MMA3001 Dataset 3 project guidance

[3] Ultralytics, “YOLOv8,” *Ultralytics Documentation*, accessed Oct. 14, 2026.

[4] M. L. Ali and Z. Zhang, “The YOLO Framework: A Comprehensive Review of Evolution, Applications, and Benchmarks in Object Detection,” *Computers*, vol. 13, no. 12, Art. no. 336, 2024, doi: 10.3390/computers13120336.

[5] R. Padilla, W. L. Passos, T. L. B. Dias, S. L. Netto, and E. A. B. da Silva, “A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit,” *Electronics*, vol. 10, no. 3, Art. no. 279, 2021, doi: 10.3390/electronics10030279.

Appendix A: Initial V4 Baseline Investigation

A.1 Purpose of the Initial Baseline

An initial baseline object-detection model was developed using **Dataset Version 4 (V4)** and the YOLOv8n architecture. This experiment was used to establish initial model performance and investigate the suitability of the supplied five-class dataset before subsequent model development.

The V4 dataset contained the following classes:

1. loose-meat
2. packaging-error
3. twisted-meat
4. unsealed
5. wrinkle

YOLOv8n was selected as the initial baseline because it is the smallest YOLOv8 model investigated in this project, providing relatively low computational cost while still supporting multi-class object detection. The later project development retained YOLOv8n after comparison with the larger YOLOv8s architecture.

A.2 Dataset Preparation

Before training, the V4 dataset was audited to reduce source-image leakage between the training, validation and test subsets. Augmented images originating from the same source frame were identified using the filename prefix preceding `.rf.`. Overlapping source identifiers were removed while prioritising the test split.

One wrinkle source was additionally reserved for validation because the original validation and test distributions provided insufficient independent wrinkle coverage. The cleaned V4 split and class distribution are summarised in **Table A.1**.

Table A.1. Cleaned V4 dataset distribution

Split	Images	Loose meat	Packaging error	Twisted meat	Unsealed	Wrinkle
Training	3310	66	1679	72	1052	15
Validation	122	3	34	3	80	3
Test	80	3	23	4	49	0

A major limitation was that **the test set contained no labelled wrinkle examples**. Therefore, the final V4 test evaluation could not establish wrinkle-specific recall or average precision.

A.3 Baseline Training Configuration

The baseline training configuration is summarised in **Table A.2**.

Table A.2: V4 YOLOv8n baseline training configuration

Parameter	Baseline setting
Model	YOLOv8n
Epochs	30
Input image size	640 px
Batch size	16
Random seed	42
Device	NVIDIA Tesla T4 GPU
Mosaic augmentation	Default enabled
Optimiser	Ultralytics automatic selection
Number of classes	5

This experiment was intended to provide a reference model rather than an extensively optimised configuration. Later development investigated mosaic augmentation, image resolution, learning rate and training duration separately.

A.4 V4 Baseline Test Performance

The trained V4 baseline was evaluated on the cleaned test split of **80 images containing 79 labelled defects**. Overall test performance is summarised in **Table A.3**, with class-specific results provided in **Table A.4**.

Table A.3: Overall V4 YOLOv8n test performance

Metric	Precision	Recall	mAP50	mAP50–95
V4 YOLOv8n baseline	**56.5%**	**49.0%**	**53.3%**	**26.6%**

The results indicate that the baseline could detect and localise several defect types but still missed approximately half of the labelled defects overall.

Per-class performance

Table A.4: Class-specific V4 YOLOv8n test performance

Class	Instances	Precision	Recall	mAP50	mAP50–95
Loose meat	3	100.0%	60.6%	73.1%	32.9%
Packaging error	23	49.8%	51.8%	49.7%	12.4%

Class	Instances	Precision	Recall	mAP50	mAP50–95
Twisted meat	4	0.0%	**0.0%**	6.1%	2.5%
Unsealed	49	76.0%	83.7%	84.4%	58.6%
Wrinkle	0	—	—	—	—

The strongest performance was obtained for **unsealed packaging**, whereas **twisted meat was not successfully detected in the test set**. Wrinkle performance could not be assessed because no labelled wrinkle instances were present in the test split. The normalised validation confusion matrix in **Figure A.1** further illustrates the class-specific failure pattern.

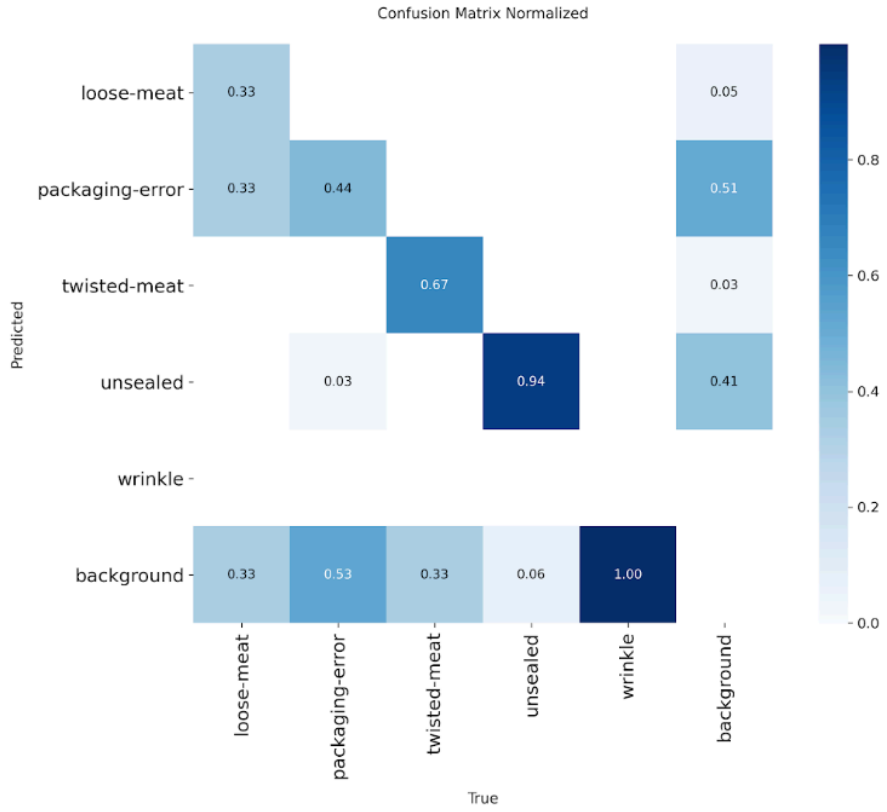


Figure A.1. Normalised validation-set confusion matrix for the V4 YOLOv8n baseline.

A.5 Wrinkle-Class Investigation

The wrinkle class presented a particular limitation in V4. Only a small number of independent wrinkle sources were available, and the exported preprocessing used centre cropping. Inspection of wrinkle samples showed that cropping could remove substantial image context and, in some cases, truncate the annotated defect region.

Although wrinkle examples were available for training and a small exploratory validation set was created, the test set contained no wrinkle labels. Consequently, the V4 baseline could not provide a meaningful independent measure of wrinkle generalisation.

This limitation should therefore be interpreted as a **dataset and preprocessing limitation**, rather than evidence that the YOLO architecture itself is incapable of detecting wrinkles. Representative centre-cropped labelled samples are shown in **Figure A.2**, illustrating the loss of tray context and annotations positioned close to image boundaries.

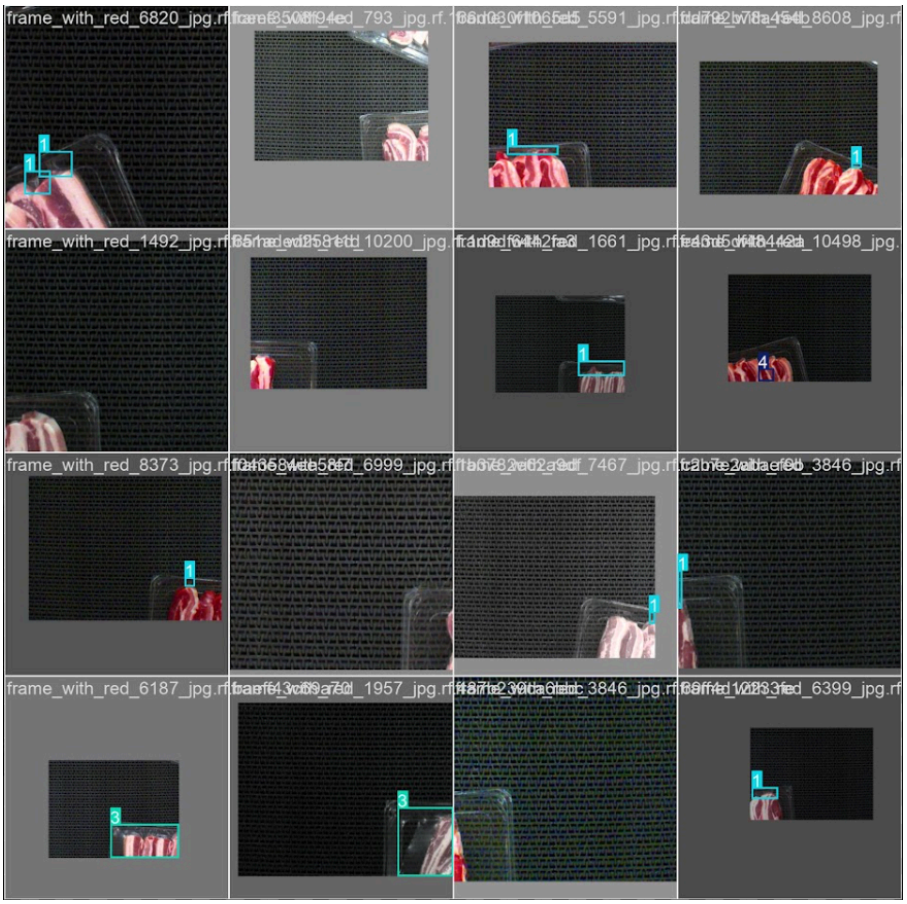


Figure A.2. Examples of V4 centre-cropped labelled samples illustrating reduced tray and defect context.

A.6 Baseline Observations

The main findings from the initial V4 experiment were:

- YOLOv8n provided a functioning object-detection baseline.
- Unsealed packaging was substantially easier to detect than the minority defect classes.
- Packaging-error detection remained moderate.
- Twisted-meat detection was particularly weak.
- The dataset was strongly class-imbalanced.
- The wrinkle class had insufficient independent validation/test coverage.
- Centre cropping could remove useful defect and tray context.
- Dataset preparation therefore had a significant influence on model performance.

These observations motivated subsequent investigation of alternative dataset versions and preprocessing methods.

Appendix B: Dataset V2 and Model-Capacity Investigation

B.1 Purpose of the Version 2 Investigation

Dataset Version 2 was investigated after the initial Version 4 experiment to evaluate a four-class object-detection problem without the poorly represented wrinkle class. The target classes were:

- loose-meat
- packaging-error
- twisted-meat
- unsealed

This dataset therefore aligned directly with the four-class defect-detection scope considered during subsequent model development.

Version 2 was also used to investigate whether increasing model capacity from **YOLOv8n** to **YOLOv8s** would improve defect-detection performance.

B.2 Dataset Preparation

The original Version 2 dataset was checked for augmented images originating from the same source frame. Source identifiers were obtained from the filename prefix preceding `.rf.` and overlapping sources were removed between training, validation and test subsets. The cleaned V2 split is summarised in **Table B.1**.

Table B.1: Cleaned V2 dataset split

Split	Training	Validation	Test
Images	1,935	119	80

No additional wrinkle holdout was required because Version 2 contained only the four selected defect classes.

The cleaned training set contained substantially more unsealed and packaging-error annotations than loose-meat and twisted-meat examples, indicating significant class imbalance. This imbalance was retained as a dataset limitation rather than manually altering the lecturer-provided annotations.

The source-identification approach also has a limitation: visually duplicated images with different filename identifiers may not be identified by the `.rf.` prefix method.

B.3 YOLOv8n Baseline Configuration

The Version 2 baseline used the pretrained **YOLOv8n** object-detection model. The V2 baseline configuration is summarised in **Table B.2**.

Table B.2: V2 YOLOv8n baseline training configuration

Parameter	Setting
Architecture	YOLOv8n
Epochs	30
Input resolution	640 px
Batch size	16
Random seed	42
Number of classes	4
Dataset	Cleaned Version 2

The notebook retains the previously trained model weights rather than automatically retraining the model. Therefore, the notebook records and reproduces the experimental configuration, while the saved weights provide the original trained model.

B.4 YOLOv8n Test Performance

The trained V2 YOLOv8n baseline was evaluated on **80 test images containing 79 labelled defect instances**. Overall test performance is shown in **Table B.3**, with class-specific results provided in **Table B.4**.

Table B.3: Overall V2 YOLOv8n test performance

Metric	Precision	Recall	mAP50	mAP50–95
V2 YOLOv8n	**77.52%**	**43.92%**	**49.85%**	**24.34%**

Table B.4: *Class-specific V2 YOLOv8n test performance*

Class	Instances	Precision	Recall	mAP50	mAP50–95
Loose meat	3	72.0%	33.3%	55.5%	21.8%
Packaging error	23	58.1%	60.9%	55.4%	17.6%
Twisted meat	4	100%*	**0%**	3.1%	0.7%
Unsealed	49	80.0%	81.5%	85.5%	57.3%

*The reported twisted-meat precision should not be interpreted as successful performance because **none of the four true twisted-meat defects were detected**. Its recall was therefore zero. The extremely small test sample also prevents strong conclusions regarding this class.

Overall, the V2 model performed most consistently on **unsealed packaging**, while twisted meat remained the most difficult class.

B.5 YOLOv8s Model-Capacity Experiment

A larger **YOLOv8s** model was subsequently trained using the same cleaned V2 dataset to investigate whether greater model capacity would improve defect detection. The controlled architecture settings are summarised in **Table B.5**, while the validation metrics and detection observations are compared in **Tables B.6** and **B.7**, respectively.

Table B.5: Controlled training settings for YOLOv8n and YOLOv8s

Parameter	YOLOv8n	YOLOv8s
Epochs	30	30
Resolution	640 px	640 px
Batch size	16	16
Seed	42	42
Dataset	V2	V2

For a valid architecture comparison, **validation performance should be compared with validation performance**, rather than comparing the YOLOv8s validation results against the YOLOv8n test set. The recorded validation comparison was:

Table B.6: V2 validation performance: YOLOv8n vs YOLOv8s

Validation metric	YOLOv8n	YOLOv8s
Precision	**63.2%**	52.3%
Recall	**47.8%**	47.1%
mAP50	51.9%	**54.8%**
mAP50–95	**30.8%**	27.3%

The uploaded V2 results confirm the YOLOv8s validation values of **52.3% precision, 47.1% recall, 54.8% mAP50 and 27.3% mAP50–95**. The confusion-matrix investigation also showed:

Table B.7: V2 validation detection observations: YOLOv8n vs YOLOv8s

Validation observation	YOLOv8n	YOLOv8s
Packaging errors correctly detected	8 / 34	**15 / 34**
Twisted meat correctly detected	0 / 3	0 / 3
Unmatched false-positive detections	**20**	41

The corresponding normalised validation confusion matrices are shown in **Figure B.1**. Both architectures missed all three twisted-meat validation instances, while YOLOv8s detected more packaging errors but produced more unmatched false positives.

Therefore, increasing model capacity improved detection of some packaging errors and slightly increased mAP50, but this came at the cost of **lower precision, lower mAP50–95 and substantially more false-positive detections**. Twisted-meat detection remained unresolved.

This supported retaining **YOLOv8n** as the main architecture rather than moving to YOLOv8s.

B.6 Image-Preprocessing Limitation

Further visual inspection identified an important limitation of Version 2: its **centre-crop preprocessing could remove parts of the tray and labelled defects located near image boundaries**.

This was particularly relevant to twisted-meat examples. Several images contained only partial views of the affected meat region after preprocessing, reducing the visual context available to the detector.

This finding suggested that the poor minority-class performance could not necessarily be solved simply by increasing model size.

A subsequent investigation of Dataset Version 1 showed that its fitted-image preprocessing preserved more of the complete tray and defect regions. This provided the motivation to investigate Version 1 for further model development.

B.7 Key Findings from Version 2

The Version 2 investigation established that:

- removing the wrinkle class produced a clearer four-class detection problem;
- YOLOv8n achieved high test precision but relatively low overall recall;
- unsealed packaging was the most reliably detected class;
- twisted-meat detection remained extremely weak;
- YOLOv8s improved some packaging-error detection but introduced substantially more false positives;
- increased neural-network capacity alone did not resolve the dataset limitations;
- centre cropping could remove important defect context; and
- these findings justified retaining **YOLOv8n** while investigating **Version 1 preprocessing** for subsequent development.

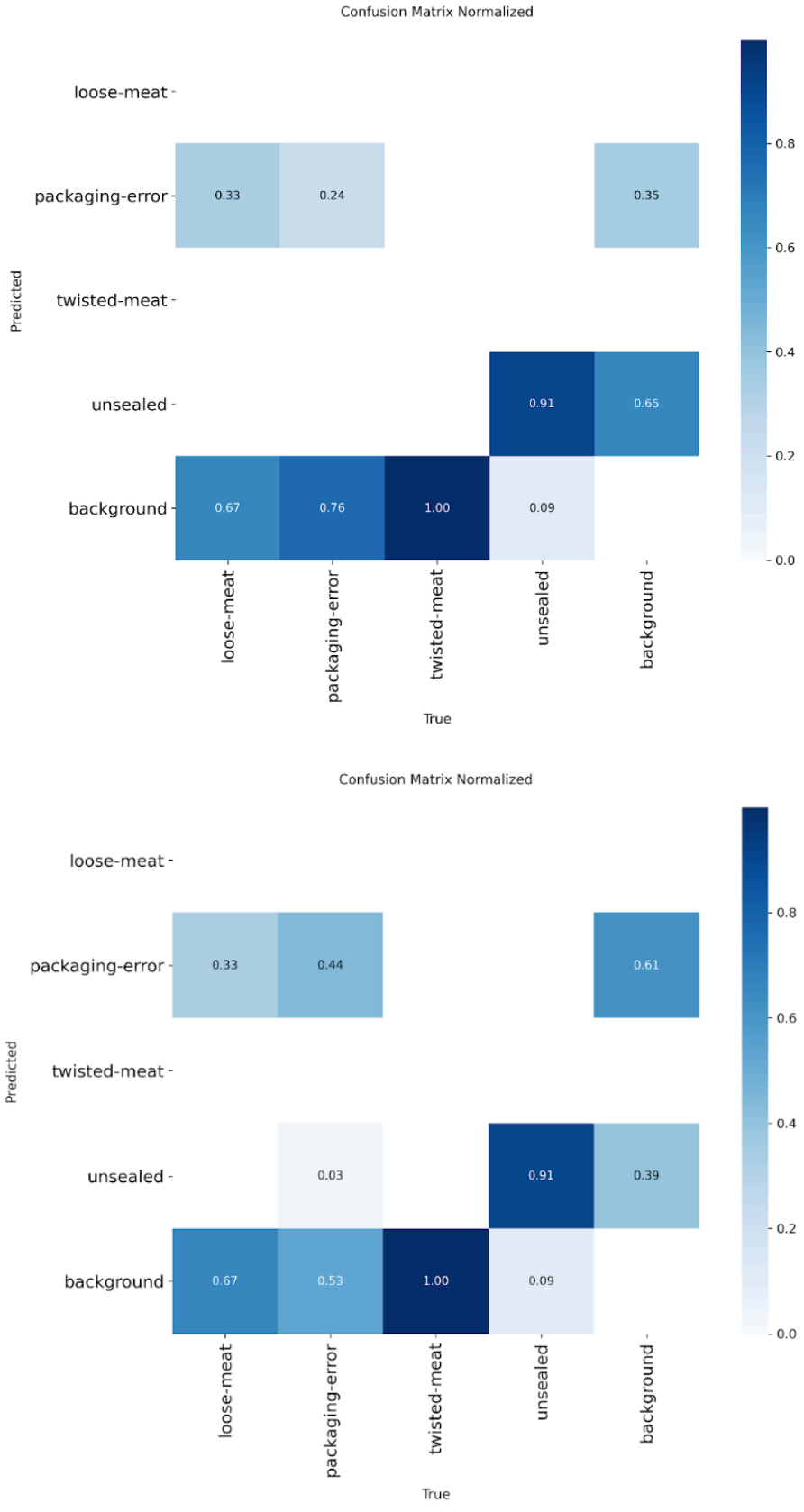


Figure B.1. Normalised V2 validation confusion matrices for YOLOv8n (top) and YOLOv8s (bottom).

Appendix C: Dataset Version 1 Development and Optimisation

C.1 Purpose of the Version 1 Investigation

Dataset Version 1 (V1) was investigated after the Version 2 study identified limitations associated with centre-cropped preprocessing. V1 used fitted-image preprocessing, which preserved more of the complete pork tray and surrounding defect context. It therefore provided a more suitable basis for subsequent model development.

V1 contained four target defect classes: loose-meat, packaging-error, twisted-meat and unsealed.

Before model development, augmented images originating from the same source were identified using the filename prefix preceding `.rf.`. Cross-split overlap was removed while preserving the test split.

After cleaning, the dataset contained:

Table C.1. Cleaned V1 dataset distribution

Split	Images	Background images	Loose meat	Packaging error	Twisted meat	Unsealed
Training	1,935	29	139	931	91	1,107
Validation	119	0	5	57	5	87
Test	80	1	3	37	7	52

The dataset remained strongly class-imbalanced. Loose-meat and twisted-meat contained substantially fewer labelled instances than packaging-error and unsealed, particularly in the validation and test subsets. Class-specific results for these minority classes should therefore be interpreted cautiously.

C.2 V1 YOLOv8n Baseline

A YOLOv8n model was trained on the cleaned V1 dataset to establish a reference configuration for subsequent optimisation.

Table C.2. V1 YOLOv8n baseline training configuration

Parameter	Baseline setting
Architecture	YOLOv8n
Epochs	30
Input resolution	640 px
Batch size	16

Parameter	Baseline setting
Random seed	42
Mosaic augmentation	Default enabled
Number of classes	4

The baseline model achieved the following results on the 119-image validation set:

Table C.3. V1 baseline validation performance

Metric	Precision	Recall	mAP50	mAP50–95
V1 baseline	77.6%	46.8%	68.3%	32.3%

The baseline therefore provided relatively high precision, but its recall of 46.8% indicated that a substantial proportion of labelled defects were still missed. Since reducing missed defects was an important engineering objective, subsequent experiments focused particularly on improving recall without causing an unacceptable reduction in overall detection quality.

C.3 Mosaic-Augmentation Investigation

The first controlled optimisation experiment investigated the effect of mosaic augmentation. Mosaic augmentation combines multiple training images into a composite image, which can increase visual diversity but can also change the spatial context surrounding the pork trays and defects.

Because the preceding dataset investigation indicated that preserving the original tray context was important, a second YOLOv8n model was trained with mosaic augmentation disabled. The remaining major settings were kept unchanged.

Table C.4. Baseline and no-mosaic controlled training settings

Parameter	Baseline	No mosaic
Architecture	YOLOv8n	YOLOv8n
Epochs	30	30
Input resolution	640 px	640 px
Batch size	16	16
Random seed	42	42
Mosaic augmentation	Enabled/default	Disabled

Both models were evaluated on the same 119 validation images containing 154 labelled defect instances using conf=0.001 and iou=0.7.

Table C.5. Baseline vs no-mosaic validation performance

Validation metric	Baseline	No mosaic	Difference
Precision	**77.6%**	64.1%	−13.5 percentage points
Recall	46.8%	**62.3%**	+15.5 percentage points
mAP50	68.3%	**69.5%**	+1.2 percentage points
mAP50–95	**32.3%**	32.1%	−0.2 percentage points

Disabling mosaic therefore produced a clear precision-recall trade-off. Precision decreased substantially, but recall increased from 46.8% to 62.3%, while mAP50 increased slightly and mAP50–95 remained approximately unchanged.

The class-specific validation recall also changed:

Table C.6. Class-specific validation recall: baseline vs no-mosaic

Class	Baseline recall	No-mosaic recall
Loose meat	80.0%	80.0%
Packaging error	13.8%	26.3%
Twisted meat	20.0%	60.0%
Unsealed	73.3%	82.8%

The apparent twisted-meat improvement should be interpreted cautiously because the validation set contained only five twisted-meat instances. Nevertheless, the overall recall improvement supported carrying the no-mosaic configuration forward for further investigation.

C.4 Confidence Operating Point

The F1-confidence curves were examined to identify representative operating points for the baseline and no-mosaic models. The baseline F1 curve peaked at a higher confidence threshold, while the no-mosaic model reached its useful operating region at a lower threshold.

For the selected no-mosaic model, a confidence threshold of approximately 0.201 was retained for the inspection demonstration. At this operating point, the model prioritises retaining more potential defects for inspection, accepting a corresponding increase in false-positive detections.

This threshold is an inference operating point and is separate from the low confidence threshold used to calculate the validation mAP metrics.

C.5 Input-Resolution Investigation

A further controlled experiment investigated whether increasing the training input resolution from 640 to 960 pixels improved detection of smaller or more difficult defects. A new YOLOv8n model was trained for 30 epochs at 960-pixel resolution while retaining the no-mosaic configuration, batch size 16 and random seed 42.

The validation results were compared with the selected 640-pixel no-mosaic configuration:

Table C.7. Validation performance at 640 px and 960 px

Metric	640 px no mosaic	960 px no mosaic
Precision	**64.1%**	58.3%
Recall	**62.3%**	55.3%
mAP50	**69.5%**	57.7%
mAP50–95	32.1%	32.1%
Packaging-error recall	26.3%	**38.6%**

Increasing the training resolution to 960 pixels improved packaging-error recall from 26.3% to 38.6%, suggesting that the additional image detail benefited this class. However, overall precision decreased from 64.1% to 58.3%, recall decreased from 62.3% to 55.3%, and mAP50 decreased from 69.5% to 57.7%. mAP50–95 remained approximately unchanged at 32.1%.

The higher-resolution model therefore produced a class-specific benefit rather than a consistent overall improvement. Because the project prioritised overall defect recall while also considering mAP50, the 640-pixel no-mosaic configuration was retained for subsequent development.

C.6 Learning-Rate Investigation

Learning rate was subsequently investigated using AdamW while retaining the YOLOv8n architecture, 640-pixel input resolution, batch size 16, seed 42 and disabled mosaic augmentation.

Three configurations were evaluated:

Table C.8. AdamW learning-rate experiment configurations

Experiment	Learning rate	Epochs
LR A	0.00125	30
LR B	0.0005	30
LR C	0.0005	50

The original 640-pixel no-mosaic model was retained as a reference. Validation results were:

Table C.9. AdamW learning-rate validation results

Model	Precision	Recall	mAP50	mAP50–95
Original no mosaic	64.1%	**62.3%**	**69.5%**	32.1%
LR 0.00125, 30 epochs	70.0%	49.5%	58.1%	29.1%
LR 0.0005, 30 epochs	**73.6%**	55.6%	60.3%	**33.5%**
LR 0.0005, 50 epochs	65.3%	57.1%	63.6%	31.4%

The controlled 30-epoch learning-rate comparison showed that reducing the learning rate from 0.00125 to 0.0005 improved all four validation metrics. Precision increased from 70.0% to 73.6%, recall from 49.5% to 55.6%, mAP50 from 58.1% to 60.3%, and mAP50–95 from 29.1% to 33.5%.

However, neither learning-rate experiment exceeded the original no-mosaic model in the two metrics given greatest importance to model selection: recall and mAP50. The original no-mosaic configuration retained the highest recall at 62.3% and the highest mAP50 at 69.5%.

The epoch-wise validation mAP50 trends are shown in **Figure C.1**, with the 0.0005 learning-rate run generally achieving stronger mAP50 during the controlled 30-epoch comparison. **Figure C.2** shows the corresponding validation box and classification-loss trends during training.

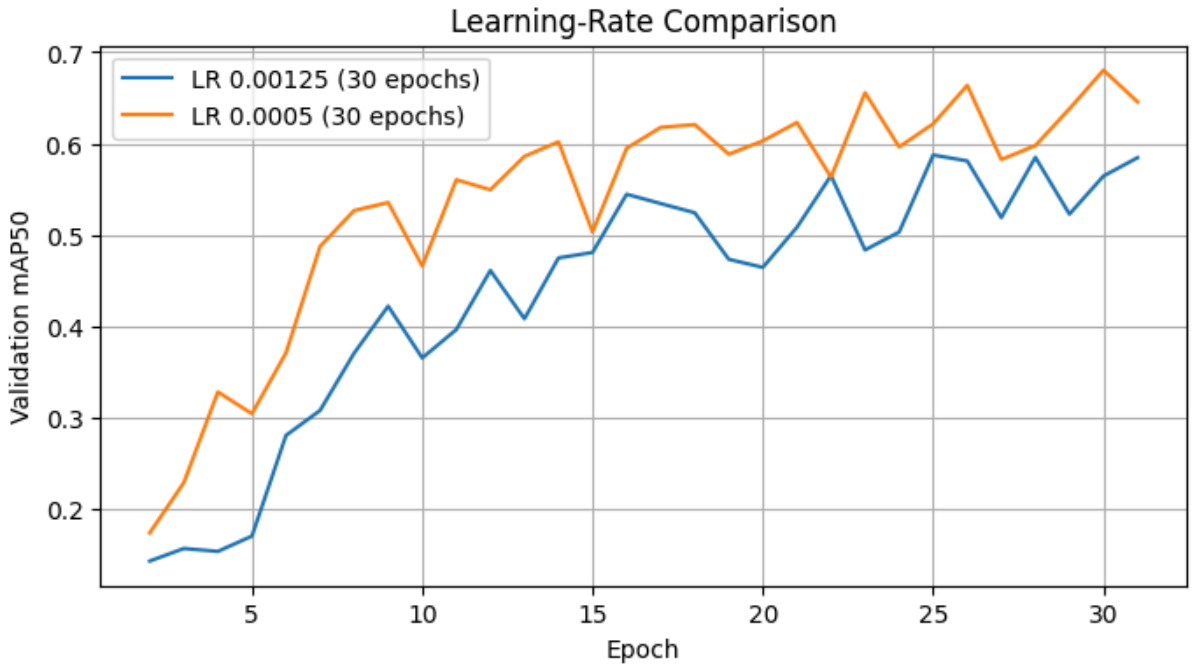


Figure C.1. Validation mAP50 across epochs for the 0.00125 and 0.0005 AdamW learning-rate experiments.

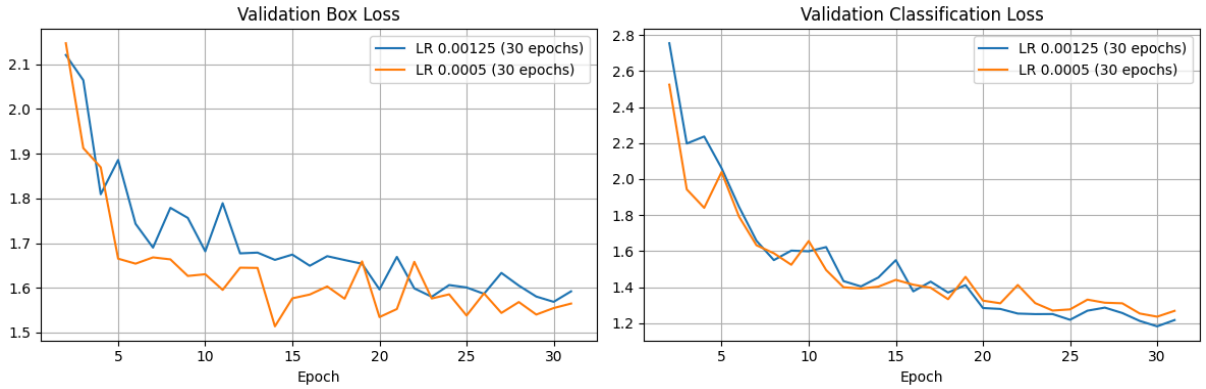


Figure C.2. Validation box loss and classification loss for the AdamW learning-rate experiments.

C.7 Training-Duration Investigation

The 0.0005 learning-rate configuration was extended from 30 to 50 epochs to determine whether additional training improved model performance.

Compared with the corresponding 30-epoch model, extending training to 50 epochs increased recall from 55.6% to 57.1% and mAP50 from 60.3% to 63.6%. However, precision decreased from 73.6% to 65.3%, while mAP50–95 decreased from 33.5% to 31.4%.

Longer training therefore produced another trade-off rather than a consistent improvement. The additional epochs did not provide sufficient evidence to replace the original 640-pixel no-mosaic configuration.

C.8 Model Selection

Across the V1 experiments, the 640-pixel YOLOv8n model with mosaic augmentation disabled provided the strongest validation recall and mAP50 among the configurations investigated.

Table C.10. V1 model-selection summary

Configuration	Main observation	Retained?
V1 baseline, 640 px	Higher precision but lower recall	No
V1 no mosaic, 640 px	Highest validation recall and mAP50	**Yes**
V1 no mosaic, 960 px	Packaging-error recall increased to 38.6%, but overall recall decreased to 55.3% and mAP50 to 57.7%	No
LR 0.00125, 30 epochs	Lower recall and mAP50 than reference	No
LR 0.0005, 30 epochs	Higher precision and mAP50–95, but lower recall and mAP50 than reference	No

Configuration	Main observation	Retained?
LR 0.0005, 50 epochs	Recall and mAP50 improved over the 30-epoch LR model, but precision and mAP50–95 decreased	No

The selected model should therefore be described as the **validation-selected inspection model**, rather than the universally best-performing model. Its selection reflected the engineering priority of reducing missed defects.

The selected model checkpoint used for the final inspection demonstration was: **V1/no_mosaic_640_best.pt**

C.9 Independent Test-Set Evaluation

After model selection, the no-mosaic model was evaluated on the V1 test set containing 80 images and 99 labelled defect instances.

Table C.11. V1 baseline and selected no-mosaic test performance

Test metric	V1 baseline	Selected no-mosaic model
Precision	**72.1%**	52.8%
Recall	51.8%	**52.4%**
mAP50	**58.2%**	54.4%
mAP50–95	**24.1%**	22.6%

The selected no-mosaic model achieved only a small increase in overall test recall relative to the baseline, while the recorded baseline results had higher precision, mAP50 and mAP50–95.

The stored baseline and no-mosaic test evaluations did not use explicitly identical confidence configurations, so these values should be interpreted as an observation of test behaviour rather than as a perfectly controlled head-to-head comparison.

More importantly, the substantial validation recall improvement obtained by removing mosaic did not transfer to the test set to the same extent. This demonstrates that model improvements observed during validation may not fully generalise to unseen data.

The result supports treating the developed model as a proof-of-concept pre-screening or decision-support system rather than a validated autonomous quality-control system.

C.10 Version 1 Findings

Dataset Version 1 provided a more suitable basis for development because it preserved more of the original tray context than the centre-cropped Version 2 dataset. Disabling mosaic substantially increased validation recall and was therefore carried forward. Higher input

resolution, learning-rate adjustment and additional training epochs produced useful metric trade-offs but did not consistently outperform the original 640-pixel no-mosaic configuration under the project priority of reducing missed defects.

The final test results also showed that the validation improvement did not generalise fully. This highlights the importance of independent evaluation, class-specific analysis and cautious interpretation when only a limited number of minority-class examples are available.

As shown in **Figure C.3**, the no-mosaic model reached its strongest F1 region at a substantially lower confidence than the baseline. A confidence threshold of approximately 0.201 was therefore retained for the inspection demonstration.

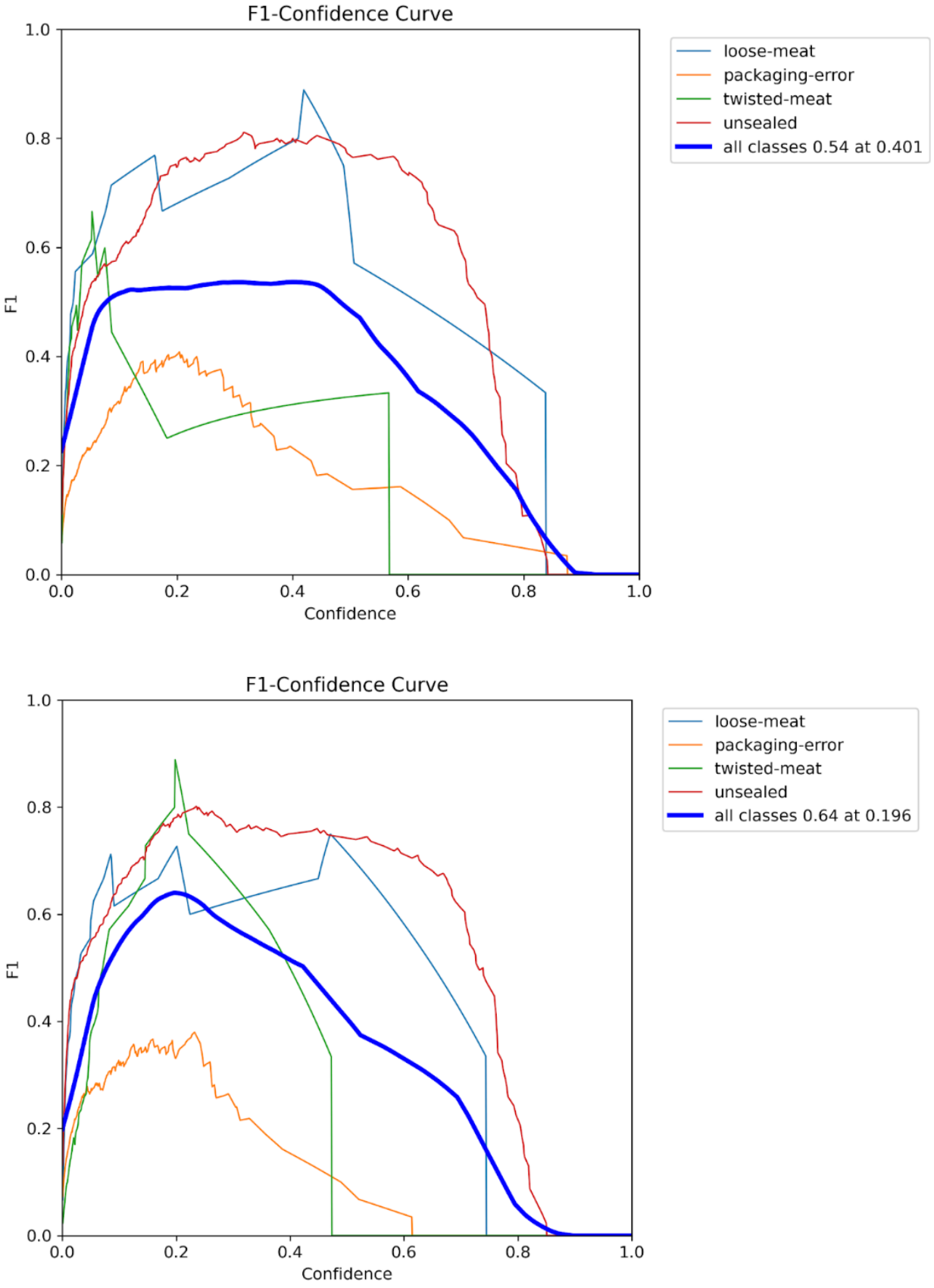


Figure C.3. F1-confidence curves for the V1 baseline (top) and no-mosaic model (bottom).